Combining Wearable Sensors and Convolutional Neural Networks in Soccer Training Drill Development

29 April 2024

Word Count: 4,969

Abstract

The combination of wearable sensors and convolutional neural networks has been researched in many classifications and other tasks. However, this combination's feasibility has not been answered for the field of soccer action classification. Due to this, soccer coaches and players would benefit from a combination that can automatically classify and highlight when these actions occur. Using a simple convolutional neural network architecture and wearable sensor made using an Arduino, accelerometer, and gyroscope, the practicability of this combination was determined. After testing and training this combination using a soccer training circuit, the combination was determined not to be feasible. This finding implicates the need for more data abstraction during collection for wearable devices, as this will lead to more data for the convolutional neural network to use as input. This will increase its classification accuracy for future applications.

Introduction

Wearable sensors have become a leading force in the sports world by increasing the data analyzation capabilities of coaches and also by adding billions of dollars to the sports analyzation market. In fact, Grand View Research (2023) found that its 2022 global market was valued at 61.3 billion US Dollars and now it is expected to have a compounded annual growth rate of 14.6 percent from 2023 to 2030 (p. 1). Many methods are used in order to analyze trillions of data points, despite being time consuming and therefore not efficient. As a result, new techniques, specifically with machine learning, have been developed to optimize data analysis.

Swathikan Chidambaram, from the Institute of Global Health Innovation at the Imperial College of London, et al. (2022) defines machine learning as "a subset of AI methodology that uses data to classify, predict or gain useful inference without explicit instructions" (p. 2). Combined with wearable sensors, machine learning can be a viable form of data analysis that soccer coaches can utilize when designing training drills. Currently, no such method has been designed to test the feasibility of machine learning powered by wearable sensors in soccer. As a result, it is necessary to create a device that examines the output of machine learning using data collected from wearable sensors in order to measure the feasibility of this technique.

Background

Convolutional neural networks are a relatively new type of machine learning, and therefore, many individuals are unfamiliar with the term. Quan Zhang (2023), professor of Electronic & Information Engineering at Suzhou University of Science and Technology, describes a convolutional neural network (CNN) as a category of neural networks (a computer program that is based off the human brain) that utilizes convolutional layers in between the input and output of the algorithm. A convolutional layer extracts initial features from the input layer.

The idea of CNNs came from human biology, specifically the biology of the visual cortex. Some neurons in the brain only respond when the cortex focuses on edges of the object being viewed, whether these edges are horizontal or vertical, while other neurons focus on other aspects of the object. All of the neurons, however, work together to create a visual representation of an object or setting (p. 434). This idea of focusing on particular components of a system is the basis for CNNs.

Moreover, Ryan T. Li (2016), MD, and his colleagues claim that *wearable sensors* provide a method of monitoring real-time physiologic and movement parameters during training and competitive sports (p. 2). They can be used to recognize position specific patterns in movement, screen for potential injuries, and help coaches develop better training plans. Wearable sensors can be used to collect and store data, which can then be interpreted by a CNN. The combination of CNNs and wearable sensors offers a new technique that can analyze soccer performance and direct coaches in their design of training drills.

Literature Review

Application of CNNs

CNNs have a powerful capacity to analyze data. In many cases, they outperform other techniques. Hayder Hasan and his colleagues, all from the engineering department at Putra Malaysia University, compared a CNN to a Support Vector Machine (SVM), which is common in speech recognition. Hasan et al. (2019) evaluated the effectiveness of both algorithms for hyperspectral image classification, which are images that have corresponding colors based off of an electromagnetic spectrum. They found that in most test cases, CNNs outperformed linear and radial basis styled SVMs, reaching a maximum classification accuracy of 98.72 percent (Hasan et al., 2019, pg. 9). In order to explain why Hasan found so much success in his method, it is

necessary to examine this method from a data processing perspective. This will show how Hasan successfully used the strengths of a CNN to show its dominance over other algorithms. Hongmei Ma and Xioafeng Pang (2019), in their paper on CNN applications in medical data processing, found that the use of CNNs allows for “the sampling [to have a] self-adjusting function, [which] keeps it as close as possible to the distribution of the original data" (p. 118843). By staying as close as possible to the original data, a higher classification accuracy is achieved by Ma and Pang (2019). Their work “verified the diagnostic accuracy of improved convolutional neural network model, traditional neural network model, [and] neural network model” (Ma & Pang, 2019, p. 10). In this source, it explains the implementation of CNNs in the medical field of sports, while the previous sources examine CNNs in the application of hyperspectral imaging. By proving that CNNs are valuable in both fields, this paper suggests that CNNs are flexible in their application, and can be implemented into the proposed method.

One example of the way that CNNs are flexible is through its application in shot classification. This topic is extremely wide, as there are numerous different types of shots, depending on the sport. Rabia A. Minhas et al. (2019) from Capital University of Science and Technology discuss the application of CNNs in shot classification for numerous sports. They chose to use a CNN over other algorithms, specifically a conventional neural network, as it is more dense with five convolutional layers. At the end of their work, the authors calculated an average precision of 94.8% and accuracy of 94.07% on these shot classifications (p. 494). The conclusion is that the proposed framework successfully classifies the input sports video into long, medium, close-up, and out-field shots. This source, similar to that of Ma and Pang’s (2019) work, offers another way in which CNNs are applied in order to classify some action, event, or

injury. In summary, CNNs have a wide scope of application, which allows for this technology to be molded to fit many functions.

Wearable Sensor Application

Similarly, wearable sensors have brought experts in the sports-science field a new way to collect data in real time. Chidambaram et al. (2022) analyzed the applications for wearable sensors in a sports-medicine field. The paper mentions that Taylormade, a golf equipment company, has a golf database of over 500,000 swings collected by wearable sensors, which AI can use to identify proper and poor technique. In another case, AI is used in F1 to simulate race decisions and strategies (p. 9). By collecting thousands of data points using wearable sensors, AI is able to interpret this data into valuable outputs that humans can use to perfect their craft, whether this is golf, F1, or soccer. Chidabaram (2022) also notes that these technologies are still in their infancy, implying that the future of AI and neural networks is bright, but there's still more research to be done with this technology (p. 10). Chidabaram's idea of how AI can be used to enhance the value of data collected by wearable sensors is bolstered by the work of Hasan (2019), Ma and Pang (2019), as well as Minhas (2019) as all three of these papers articulate the application of CNNs into athletic application. It is also important to understand why this technology is flexible, allowing it to perform in ways that Chidbaram (2022) discussed. Li (2016) discusses the basic applications of wearable sensors. In this clinical review, they found that these devices can be used to detect position-specific patterns in movement, design more efficient sports-specific training programs for performance optimization, and screen for potential causes of injury (p. 75). Wearable sensors are both flexible and powerful when applied with the right refinement. Using the athletic application given by Chidbaram (2022), it can be inferred that the process of data collection for each sport was done using the basic function of wearable

sensors, collecting position-based data that can be used to assess risk for injury or classify technique in sports as seen from Li's (2016) work. With this data, it can then be used as an input for a CNN, where the two work in tandem to create even more powerful results.

Combination in Sports

While CNNs and wearable devices are each powerful individually, their true prowess is displayed when they are combined together. Using this combination supplies a CNN with a multitude of real time data to use. An example of this combination can be seen in the Head of Health Tech Research Cluster at the Management Center Innsbruck, Bernhard Hollaus's et al. (2020) paper on catch detection in American football. The authors designed wearable sensors that consisted of a nine degree initial measurement, a pressure sensor and a microphone. These sensors collected a data set for acceleration, flux density, spin rate, and quantized audio. This data was then inputted into an algorithm based off of a CNN (Hollaus et al., 2020, p. 5). To test their device, they conducted fifty training iterations, which had an "average classification accuracy of 93 percent on the test fold of each iteration" (p. 10). The application of this technology is not only limited to American football. Due to its flexibility, wearable devices and CNNs can be adjusted in order to fit other sports. Thomas Kautz, Doctor of Engineering at Friedrich Alexander University of Erlangen-Nürnberg, et al. (2017) analyzes this combination for activity recognition in volleyball. Using the wearable sensor, three-axial (xyz) acceleration data was recorded from thirty subjects. A *Deep Convolutional Neural Network* (DCNN) was used to classify this data into one of the actions in volleyball such as a dig or spike (Kautz et al., 2017, p. 5). The DCNN worked with the wearable sensor by collecting three-axial acceleration data as the input for the DCNN. The DCNN used this input to create an output as a classified action. The accuracy of the DCNN was compared to other algorithms. Kautz and his colleagues'

(2017) DCNN outperformed the second best method by 16 percent, reaching an accuracy of 83 percent (p. 18). This source is very similar to Hollaus and his colleague's paper, as they discuss the application of the same technology in two different sports. Using wearable sensors, both are able to collect data and use it in the same way. Both papers further the likelihood of feasibility of this method in the wide field of sports. In summary, the combination of wearable sensors and CNNs are an extremely powerful way to collect and analyze data in an efficient manner, especially in athletic applications.

Lack of Research in Soccer

Wearable sensors have had an increasing role in the wide world of sports. They have been used in a multitude of purposes for both monitoring health levels during a performance and also collecting data for lata use of gameplay analysis. Ma & Pang (2019) as well as Minhas and his colleagues (2019) show how the power of convolutional neural networks are extensive, and have the capacity to have high accuracy diagnoses and classification of actions. In combination with convolutional neural networks, wearable sensors are able to run in a near-autmous state, being able to both collect and analyze data on its own. As seen in Hollaus and his colleagues' (2020) paper in conjunction with Thomas Kautz and his colleagues' (2017) paper, this combination has immense power when it comes to classifying actions in two different sports, American football and volleyball. No such research has been conducted in the area of on-field actions in soccer. Therefore, this research aims to fill this gap by designing a wearable sensor and CNN combination that will be used to classify actions in soccer.

Method

Due to the lack of a standardized wearable sensor meant for the data required in this method, it is necessary to create a custom wearable sensor. The parameters for this design needs

to include compactness, durability, and the ability to collect both movement and angular accelerations. Compactness and durability will allow for the sensor to easily attach to the player's ankle, the location for data collection. This construction of a device follows the action research method, which aims to both simultaneously investigate and solve a problem. This is the method that Hollaus et al. (2020) used in his paper. He created a wearable device using multiple instruments, mainly an inertial measurement unit to track accelerations to collect data. This data was then used as input for his CNN to classify catches or drops (Hollaus et al., 2020, p. 5). Kautz (2017) performs a similar action research method as he and his colleagues created a three-axial wearable sensor to collect acceleration data from volleyballs. Again, this was used as an input for a DCNN to classify actions. By utilizing a similar method to Hallaus and Kautz, it is possible to approach this problem in a similar fashion. The action research method was chosen over many other methods, most notably an experimental research method, as it allows for the most freedom in a study. An experimental research design, for instance, requires specified variables which must be held constant, used independently, or used as a dependent. This design is too rigid for this paper's purpose, as there are far too many variables to account for. It would require multiple different configurations to study the change of six different variables and how it affects the outcome of a classification, nor would answer the research question in a direct manner. The action research method allows for the creation of a custom wearable sensor, tailored to collect data only necessary to action classification in soccer. It also allows users to bypass the paywall behind standardized wearable devices that can reach costs in the hundreds. Therefore, it will be able to answer the problem of the feasibility of the combination of wearable sensors and CNNs in action classification of soccer actions.

Designing the Wearable Sensor

There are two main components that constitute wearable sensors. The first component of the proposed wearable sensor is an Arduino Nano Every. This microcontroller acts as the brain of the wearable sensor, as it instructs the other module, a DFR Robot BMI160 6 Degree of Freedom (DOF) Inertial Measurement Unit (IMU), to collect and transmit data. This microcontroller was chosen for this method due to its compact structure, processing power, and affordable cost. While only being 45 millimeters long and 18 millimeters wide, the Nano Every has a clock speed of 20 Megahertz and a static random access memory (SRAM) of six kilobytes, meaning it is able to process multiple inputs quickly. Moreover, the BMI160 Six DOF IMU was chosen due to its compact nature, simple implementation, and once again, its affordability. The purpose of an IMU is to collect movement based data. In this case of the BMI160, it consists of a gyroscope to measure angular rates and an accelerometer to measure acceleration data. This sensor is able to collect six different pieces of data, three for movement based acceleration and three for angular accelerations. The two components were connected via the SDA and SCL ports of the arduino, and powered by a five volt, two point four amp USB battery. All data collected was stored on a micro SD card, which connected to the wearable device through an SD card module. One limitation to design of this wearable device was the size of the battery. While batteries that supply a minimum of five volts and a current of two amps are available, they can be quite costly to purchase batteries that fit these requirements in a small stature. See appendix A to view the software of the device, and figure 1 for the schematic of the wearable sensor.

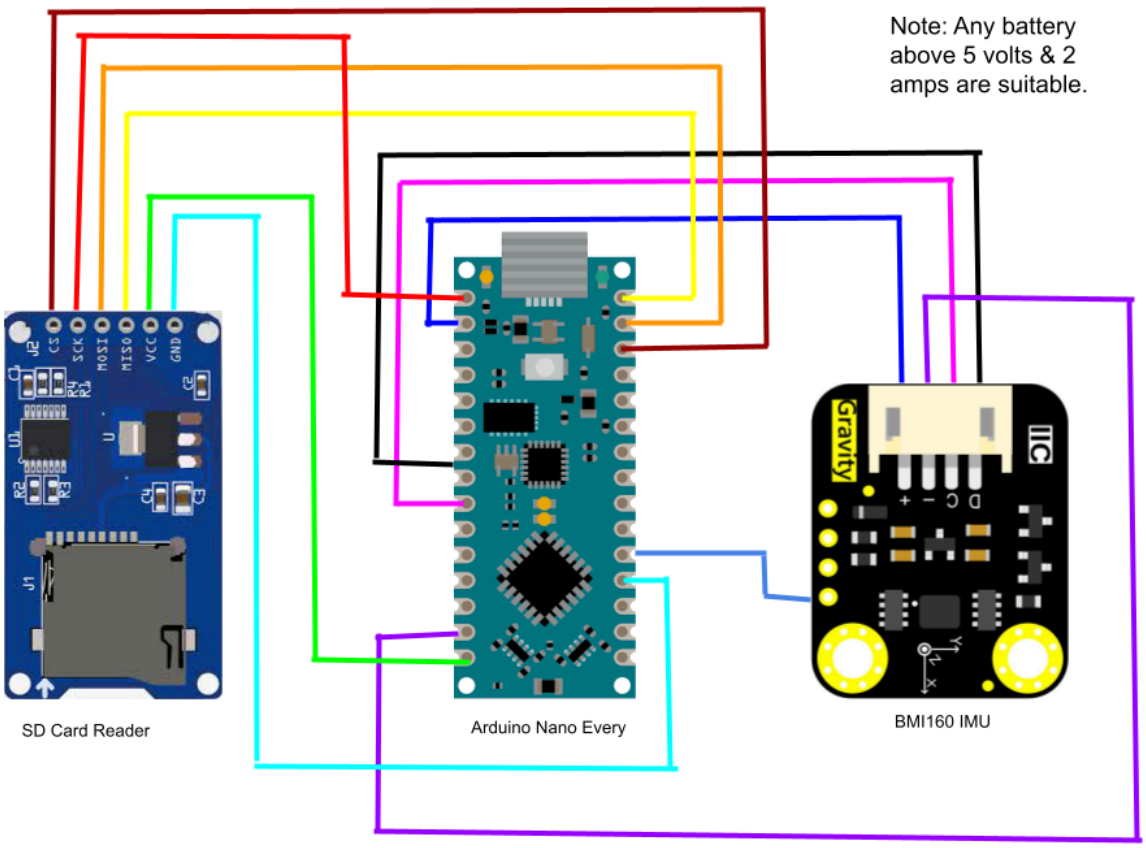


Figure 1

Designing the CNN

For the purposes of this paper, a one dimensional CNN will be created with an input shape of four hundred twenty by six. This input shape was chosen because the wearable sensor reads data at a rate of seven hertz, or seven times per second. So, 420 rows of data results in 60 seconds of data collection. Six columns were chosen as each piece of data consists of six values as mentioned above. The main structure of the CNN was made using the python coding language, visual studio code, and the Keras library. Python was chosen due to its simplicity, as well as its uniformity across the machine learning community. Visual Studio Code was also used

due to its convenience as the standard code editor on any windows device. Finally, Keras was chosen due to its prowess in developing machine learning applications. It assists in simplifying code, using its built-in function to construct layers for algorithms. Two convolutional layers are implemented, which filters the data into feature maps. Then, a max pooling layer will be used over a feature map of the data created in order to initialize the break up of data in classifications by locating local maximums in the data. A flatten layer was used to prepare the data for the dense layer. In the dense layer, an ReLU activation was used to non-lineare the data, making classification easier and more abstract. A dropout layer was added to prevent overfitting, a failure in the model that can lead to false predictions. The final layer of this model is another dense layer, which uses a softmax activation in order to provide an output that is readable to users. This will then lead to the output, which takes the form of a four by one matrix. Each value of the matrix will be a probability of a certain action being done, with all the elements summing to one. The order of the matrix is: jogging/sprinting, dribbling, passing, and shooting. See appendix A to view the software of the CNN.

Implementing the technology

In order to establish the feasibility of this technology, the combination of the wearable sensors and CNN needed to be tested. This was done through the collection of data in an organized soccer training session. For training data sets, players were instructed to take five shots, passes, and dribbles in any order. For test sets, players were instructed to take any amount of shots, passes, or dribbles in one minute. All data was stored onto a sd card, which was then input into a computer, where the CNN was located. The data was manually labeled by its action and then input into the CNN. The total training data spread was 80 percent training data and 20 percent validation. One additional data set was collected to serve as data to be predicted by the

CNN model. Each data set was intended to display the differences among each action, allowing the CNN to recognize it. For example during a shot, there were moments of high movement acceleration and low angular acceleration, due to the players moving at a high speed, but keeping their ankle locked in order to shoot well. For a pass, there were moments of acceleration below that of a shot, while also high moments of angular acceleration to the players rotating there ankle to pass the ball with the inside of their foot.

The circuit used for collecting the data was set up using a soccer goal, a rebounder, and cones. Players began at the dribbling section of the circuit, and were instructed to dribble through the cones then return the ball back to its original position. Then, they were instructed to run to the passing station, where they would pass a ball against a rebounder, then receive the ball back. Finally, the players were told to run to the shooting station, in which they took a shot, aiming for one of the corners in the goal, however, any style of placement for the shot can be used, as this does not affect the data collected. Refer to figure two for a diagram of the circuit.

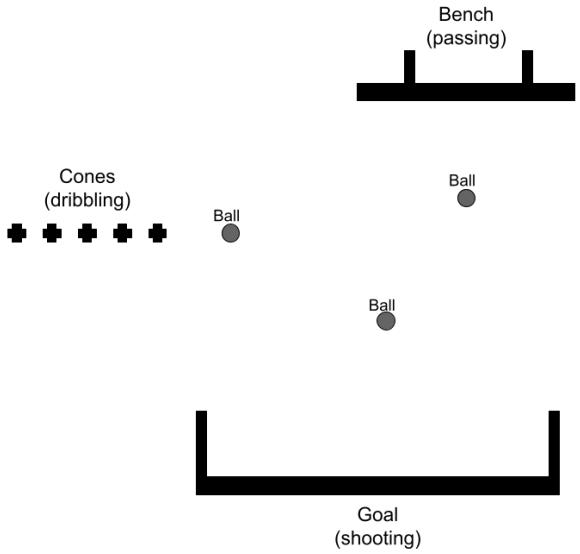


Figure 2

One of the limitations of the proposed method is the forced upon bias towards running. This bias is due to this action occurring between all other actions. During the transition, players will run in order to get to the next station. This was minimized by narrowing the length gaps in between each part of the training circuit. Another limitation to the proposed method is the size of the wearable device. The device needed to remain small in order to fit well and not inhibit a player's performance. This meant that there was a limit on the size and amount of modules that were able to fit onto the wearable device. As a result, the sensor modules mentioned previously were chosen.

Results

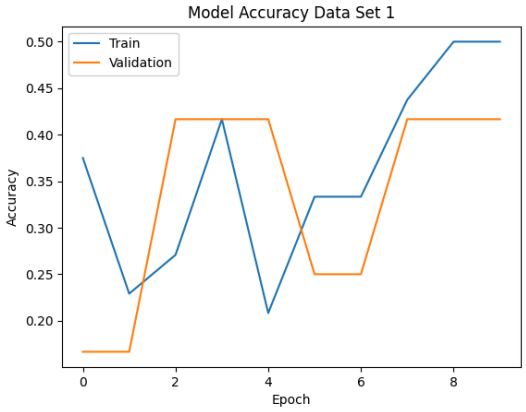
Wearable Sensor Results

The wearable device was able to successfully collect all data required for a prolonged amount of time. For each data set, more than 600 data pieces were collected, each containing the proper metrics to be inputted into the CNN. These 600 lines of data were trimmed down to account for areas of noise as well as transition periods between powering the device and beginning the actions. Areas of noise were determined to be reading from the sensor that were unfeasible. An example of noise that was cut out of the data were moments of constant, positive acceleration, with singular pieces of negative acceleration breaking them apart. These moments of negative acceleration were feasible since it is extremely unlikely that a player was able to change acceleration from a high positive number to a negative in a matter of one-seventh of a second. The highest movement based acceleration in any direction was recorded as 76.27 meters per second squared, while a minimum of 0.63 meters per second squared was recorded. The maximum angular acceleration in any direction was recorded as 2.00 radian per second squared, while a minimum value of 0.10 radians per second squared was recorded. While both of these

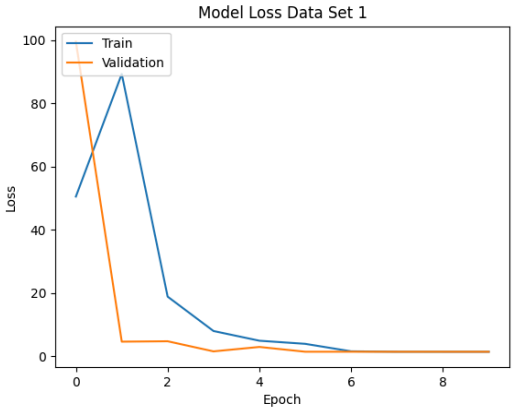
metrics have a large range, they are feasible due to high speeds while shooting, and the low speeds while walking. See Appendix B to view a 1 second example of data collected.

CNN Model Results

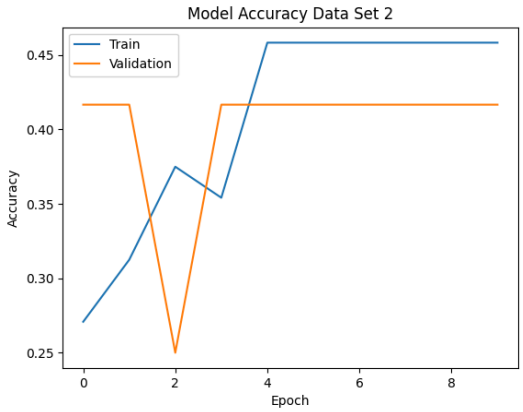
The raw data collected by the wearable sensors showed no clear distinctions when viewed from a human perspective. However, the CNN model did pick up on some slight patterns among each action. Over the course of three data sets training the CNN model, its accuracy increased while its loss, the difference between the correct predictions and the model's, decreased. Below are the graphs of the accuracy and loss across three data set iterations.



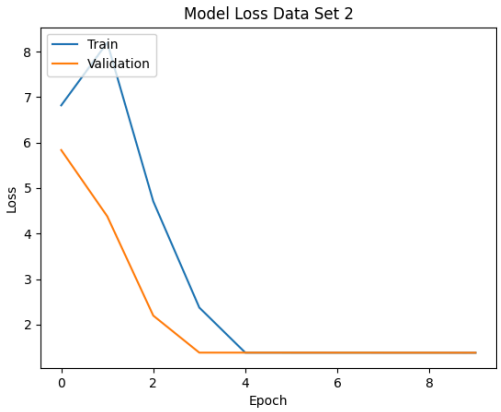
Graph 1



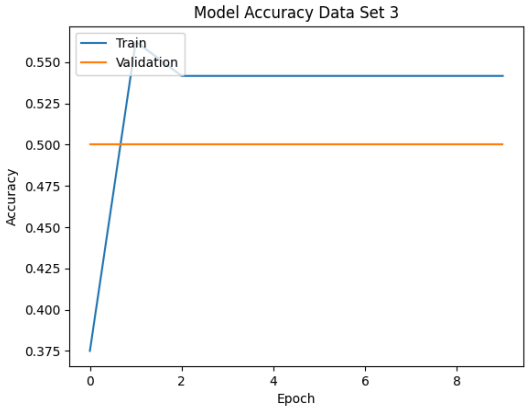
Graph 2



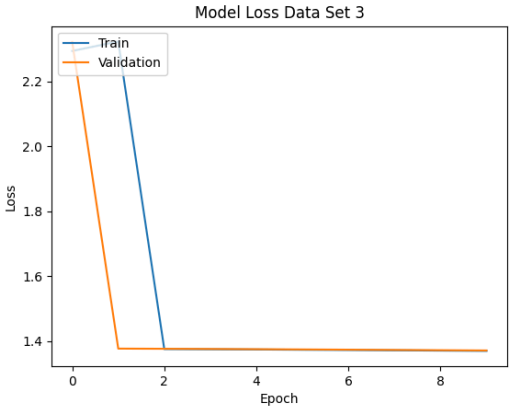
Graph 3



Graph 4



Graph 5



Graph 6

The trend described by these graphs display an increasing accuracy, peaking at around 55 percent accuracy, and a decreasing loss, reaching a minimum of 1.3. The model encountered an increase in loss when beginning training of each data set. This can be seen in graphs 2, 4, and 6 as the loss value slightly increases from epoch 0 to epoch 2. Yet, it returns back to a decreasing trend as its epochs increase. The training and validation data split both displayed similar trends across each data set, showing how the model performed near equal with and without human supervision and data labels.

After training the model using three different data sets, the model was finally tested using one test data set. The final accuracy of the model was 46.67 percent, meaning that the model predicted 28 out of the 60 possible classifications. Below is the confusion matrix for this model's predictions.

[28 0 0 0]
[20 0 0 0]
[6 0 0 0]
[6 0 0 0]

Figure 2

This matrix shows how all 60 possible classifications were considered as the first classification, meaning the CNN predicted that all instances done by the player were running or

sprinting. In the confusion matrix, the sum of each row represents the actual instances of each action. So, 28 instances of running/sprinting, 20 instances of dribbling, 6 instances of passing, and 6 instances of shooting occurred. The sum of each column represents the total number of classifications predicted by the CNN. So, 60 actions were classified as jogging or sprinting, while there were zero instances of the remaining classifications. Below are the metrics to measure the success of the model.

Metric	Precision	Recall	F1-Score	Accuracy
Value	0.0625	0.1875	0.0938	0.4667

Table 1

For precision and recall, a score close to one is ideal. Precision measures the proportion of true positive predictions to all the other positive predictions. In this case, precision is measured for the predictions for the running classification. Recall is the proportion of true positives to the sum of true positives and false negatives. It is once again for the running classification. The F1-score is the harmonic mean of recall and precision and is calculated as double the proportions of product of recall and precision to the sum of recall and precision.

Discussion

The goal of this research method was to determine the feasibility of the combinations of wearable sensors and convolutional neural networks for the purpose of action classification in soccer. My research question was fully addressed as the feasibility of this combination was determined. The majority of statistics that analyze the success of the method supported that the proposed version of the combination was not feasible for accurate classifications, despite some statistics pointing in the opposite direction.

It is important to analyze the data collected to train the CNN model as the accuracy of the CNN is directly linked to the volume and precision of this data. Despite each input shape having consistent size values, it is important to put into perspective the magnitudes of each data piece collected. Ante Rađa (2019), kinesiology specialist from the University of Split, and his colleagues conducted research about the average kicking speeds of youth soccer players, and found that the average kicking speeds of teenage, competitive soccer players was 108 kilometers per hour, or 30 meters per seconds (p. 10). In the collected data, the maximum movement acceleration was found to 76 meters per second, more than double this value. Given that the subject in this study was already moving at roughly three meters per second, it is extremely unfeasible that a change in velocity by 76 meters per second squared was experienced. On the other hand, looking at the latter half of the data collection, regarding angular accessions, the data collected was very accurate. The range of angular accelerations is very feasible as it was only from 0.10 to 2 radians per second squared. As a result of high levels of misreadings, the culmination of these two data types being input into the CNN for training likely led to faulty predictions and low accuracy for the model.

The metrics of precision, recall, F-1 score, and accuracy are good indicators of the model's success. Both the precision and recall values of this method were relatively low. The cause of these values being low was due to a high level of false-positive classifications for the jogging/sprinting actions. There were 32 false-positives, an extremely high number, given it constitutes 53 percent of all actions recorded in the test data set. The F-1 was also low, since an efficient model should perform anywhere from 0.5 to 0.9. This model score of 0.0938, is far below this score, showing how the model did not perform to expectations. Finally, the accuracy

of the model, 47 percent, also performed below expected, this is too low to be considered for complete action classification automation.

Regarding the performance of the CNN model and architecture, particularly its rate of growth, there is evidence to support the feasibility of the use of a CNN in action classification. Despite having faulty data, the CNN experienced a general trend of increasing accuracy across epochs and training sets. As seen from the graphs in the results sections, the accuracy of the model had a generally increasing trend starting from 0 percent to reaching 55 percent, ultimately peaking at this value. Also, the model's loss had a generally decreasing trend, proving that it progressed the way a standard CNN does. These trends indicate the idea that if clearer data was used, the model would have performed to a higher standard.

Limitations

The main limitation encountered in this method was with the wearable sensor. This wearable sensor was only able to collect six different data types, limiting the range of data the CNN can be trained with. This means that, in the case of noise, the lack of data diversity would indicate that the CNN would be trained mainly upon noisy data, leading to false predictions. Also, both types of data, angular and movement accelerations, were collected using the same sensor module, the BMI160 IMU, meaning that if this module collected noisy data, once again, all data types would be affected. As seen from the results, this did happen and severely inhibited the accuracy of the model. Another limitation encountered was finding a balance for the input shape requirements of the model and its complexity. Using a bigger input shape would not only require larger data sets, but it would require a more complex CNN model. This would include more convolutional, dense, and pooling layers. Not only would this increase the efficiency of the

model, but it would create a larger computing cost, limiting the adaptability of these combinations.

Implications

There are multiple implications towards this research. One of these being the importance of data diversity. Since the data in the proposed method were based on kinematic metrics, mainly acceleration, and they were collected from the same module, there was a likelihood that the model would not be able to comprehend the entire action being classified, leading to false classifications. Therefore, it is necessary to collect diverse data not limited to acceleration only. This would allow for the model to understand the problem from a larger context, leading to better predictions.

This research also shows the need to test multiple other machine learning algorithms in order to find a combination that is feasible for this application. Other models such as a support vector machine or deep reinforcement learning should be tested in the future. They may be able to recognize patterns in this form of time-series data at a more efficient rate than CNNs. Finding a feasible combination is an important research goal as it will improve the efficacy of soccer coaches and players in analyzing game footage and training film. It will allow coaches to be able to find moments in which they can review a player's specific actions without needing to spend time filtering through footage in order to find these actions manually.

References

Arduino nano every. (2024). https://docs.arduino.cc/hardware/nano-every/

BMI160 6-axis inertial motion sensor. (2024). https://wiki.dfrobot.com/Gravity__BMI160_6-Axis_Inertial_Motion_Sensor_SKU__SEN0250

Chidambaram, S., Maheswaran, Y., Patel, K., Sounderajah, V., Hashimoto, D. A., Seastedt, K. P., Mcgregor, A. H., Markar, S. R., & Darzi, A. (2022). Using artificial intelligence-enhanced sensing and wearable technology in sports medicine and performance optimisation. *Sensors*, *22*(18), 6920. https://doi.org/10.3390/s22186920

H. Ma and X. Pang, "Research and Analysis of Sport Medical Data Processing Algorithms Based on Deep Learning and Internet of Things," in IEEE Access, vol. 7, pp. 118839-118849, 2019, doi: 10.1109/ACCESS.2019.2936945.

Haladjian, J., Schlabbers, D., Taheri, S., Tharr, M., & Bruegge, B. (2020). Sensor-based detection and classification of soccer goalkeeper training exercises. *ACM Transactions on Internet of Things*, *1*(2), 1-20. https://doi.org/10.1145/3372342

Hayder Hasan, Helmi Z.M. Shafri and Mohammed Habshi. A Comparison Between Support Vector Machine (SVM) and Convolutional Neural Network (CNN) Models For Hyperspectral Image Classification. *IOP Conf. Ser.: Earth Environ. Sci.*. 2019. Vol. 357(1):012035. DOI: 10.1088/1755-1315/357/1/012035

Hollaus, B., Stabinger, S., Mehrle, A., & Raschner, C. (2020). Using Wearable Sensors and a Convolutional Neural Network for Catch Detection in American Football. *Sensors*, *20*(23), 6722. MDPI AG. Retrieved from http://dx.doi.org/10.3390/s20236722

Kautz, T., Groh, B.H., Hannink, J. *et al.* Activity recognition in beach volleyball using a Deep Convolutional Neural Network. *Data Min Knowl Disc* **31**, 1678–1705 (2017). https://doi.org/10.1007/s10618-017-0495-0

Li, R. T., Kling, S. R., Salata, M. J., Cupp, S. A., Sheehan, J., & Voos, J. E. (2016). Wearable Performance Devices in Sports Medicine. *Sports health*, *8*(1), 74–78. https://doi.org/10.1177/1941738115616917

Minhas, R. A., Javed, A., Irtaza, A., Mahmood, M. T., & Joo, Y. B. (2019). Shot Classification of Field Sports Videos Using AlexNet Convolutional Neural Network. *Applied Sciences*, *9*(3), 483. MDPI AG. Retrieved from http://dx.doi.org/10.3390/app9030483

Rađa, A., Kuvačić, G., De Giorgio, A., Sellami, M., Ardigò, L. P., Bragazzi, N. L., & Padulo, J. (2019). The ball kicking speed: A new, efficient performance indicator in youth soccer. *PloS one*, *14*(5), e0217101. https://doi.org/10.1371/journal.pone.0217101

SD card module with arduino uno. (2018). https://www.androiderode.com/sd-card-module-with-arduino-uno/#google_vignette

Wearable technology market size, share & trends analysis report by product (Head & eyewear, wristwear), by application (Consumer electronics, healthcare), by region (Asia pacific, Europe), and segment forecasts, 2023 - 2030. (2023). Grand View Research. Retrieved October 25, 2023, from https://www.grandviewresearch.com/industry-analysis/wearable-technology-market

Zhang, X., Zhang, X., & Wang, W. (2023). Convolutional Neural Network. In *Intelligent Information Processing with Matlab* (pp. 39-71). Singapore: Springer Nature Singapore.

Appendix A

Code of Wearable Device and CNN

Name and Language	Code

Wearable Device (C++)	#include "DFRobot_BMI160.h"
	#include <SPI.h>
	#include <SD.h>
	const int CS_PIN = 10;
	File file;
	String fileName = "data.txt";
	String dataPiece = "";
	DFRobot_BMI160 bmi160;
	const int8_t i2c_addr = 0x69;
	void setup(){
	Serial.begin(115200);
	delay(100);
	//init the hardware bmi160
	if (bmi160.softReset() != BMI160_OK){
	Serial.println("reset false");
	while(1);
	}

```
  //set and initialize the bmi160 i2c address
  if (bmi160.I2cInit(i2c_addr) != BMI160_OK){
    Serial.println("init false");
    while(1);
  }
  while(!Serial){};
  initializeSD();


  if(SD.exists(fileName))
  {
    writeData("\n");
  }
  writeHeader();
}
void loop(){
  int rslt;
  int16_t accelGyro[6]={0};
  //get both acceleration and gyro data from bmi160
  //parameter accelGyro is the pointer to store the data
  rslt = bmi160.getAccelGyroData(accelGyro);
  if(rslt == 0){
    for(int i=0;i<6;i++){
      if (i<3){
```

```
      //the first three are gyro data
      Serial.print(accelGyro[i]*3.14/180.0);
      writeData(String(accelGyro[i]*3.14/180.0) + ",");
      Serial.print("\t");
    }
    else if (i < 5)
    {
      //the following three data are acceleration data
      Serial.print(accelGyro[i]/16384.0);
      writeData(String(accelGyro[i]/16384.0) + ",");
      Serial.print("\t");
    }
    else
    {
      Serial.print(accelGyro[i]/16384.0);
      writeData(String(accelGyro[i]/16384.0) + "\n");
      Serial.print("\t");
    }
  }
  Serial.println();
}else{
  Serial.println("err");
}
```

```
}
void writeHeader()
{
 file = SD.open(fileName,FILE_WRITE);
 if(file)
 {
  Serial.println("--------------------\n");
  Serial.println("Writing to " + fileName + ": ");


  file.println("X_Gyro,Y_Gyro,Z_Gyro,X_Acc,Y_Acc,Z_Acc");
  file.close();
 }
 else
 {
  Serial.println("error opening " + fileName);
 }
}
void initializeSD()
{
 Serial.print("Beginning initialization of SD Card: ");
 if(!SD.begin(CS_PIN))
 {
  Serial.println("SD initialization failed");
```

	while(1){}
	}
	Serial.println("SD initialization success");
	Serial.println("------------------------\n");
	}
	void writeData(String value)
	{
	file = SD.open(fileName, FILE_WRITE);
	if(file)
	{
	file.print(value);
	file.close();
	}
	else
	{Serial.println("error opening " + fileName);
	}
	}

CNN Model (python)	from keras.models import Sequential from keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dropout from keras.utils import to_categorical from matplotlib import pyplot as plt import numpy as np

```
# Define the input shape and numbers of possible classifications
numClasses = 4
inputShape = (420, 6)
#Loading and processing all data - one hot encoding changes classes from 0-3
to [0,0,0,0]
training1Data = np.genfromtxt('TrainingOne.csv', delimiter=',')
training1Data = np.tile(training1Data, (60, 1, 1))
label1Data = np.genfromtxt('LabelOne.csv', delimiter=',')
label1ToOneHot = to_categorical(label1Data, num_classes=numClasses)
training2Data = np.genfromtxt('TrainingTwo.csv', delimiter=',')
training2Data = np.tile(training2Data, (60, 1, 1))
label2Data = np.genfromtxt('LabelTwo.csv', delimiter=',')
label2ToOneHot = to_categorical(label2Data, num_classes=numClasses)
training3Data = np.genfromtxt('TrainingThree.csv', delimiter=',')
training3Data = np.tile(training3Data, (60, 1, 1))
label3Data = np.genfromtxt('LabelThree.csv', delimiter=',')
label3ToOneHot = to_categorical(label3Data, num_classes=numClasses)
# Create the model
model = Sequential()
# Add convolutional & pooling layers
model.add(Conv1D(32, kernel_size=3, activation='relu',
input_shape=inputShape))
model.add(MaxPooling1D(pool_size=2))
```

```
model.add(Conv1D(64, kernel_size=3, activation='relu'))
model.add(MaxPooling1D(pool_size=2))
# Flatten the output for dense layers
model.add(Flatten())
# Add dense layers
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))  # Dropout for regularization
model.add(Dense(4, activation='softmax'))  # Adjust num_classes based on
your classification task
# Compile the model
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
# Print the model summary
model.summary()
#training the model
history = model.fit(training1Data, label1ToOneHot,
                    epochs=10,
                    batch_size=32,
                    validation_split=0.2)
history2 = model.fit(training2Data, label2ToOneHot,
                    epochs=10,
                    batch_size=32,
                    validation_split=0.2)
```

```
history3 = model.fit(training3Data, label3ToOneHot,
                     epochs=10,
                     batch_size=32,
                     validation_split=0.2)
model.save('CNNMain.keras') #save model so it can be used later
# Plot training & validation accuracy & loss values
#Data set 1
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy Data Set 1')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss Data Set 1')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
#Data set 2
plt.plot(history2.history['accuracy'])
```

```
plt.plot(history2.history['val_accuracy'])
plt.title('Model Accuracy Data Set 2')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
plt.plot(history2.history['loss'])
plt.plot(history2.history['val_loss'])
plt.title('Model Loss Data Set 2')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
#Data set 3
plt.plot(history3.history['accuracy'])
plt.plot(history3.history['val_accuracy'])
plt.title('Model Accuracy Data Set 3')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
plt.plot(history3.history['loss'])
plt.plot(history3.history['val_loss'])
```

	plt.title('Model Loss Data Set 3') plt.ylabel('Loss') plt.xlabel('Epoch') plt.legend(['Train', 'Validation'], loc='upper left') plt.show()

CNN Test
(python)
	from keras.models import load_model
	import numpy as np
	from sklearn.metrics import confusion_matrix, precision_score, f1_score,
	accuracy_score, recall_score
	#loading model
	model = load_model('CNNMain.keras')
	#loading and prepping data
	testData = np.genfromtxt('testData.csv', delimiter=',')
	testData = np.tile(testData, (60, 1, 1))
	testKey = np.genfromtxt('labelTest.csv',delimiter=',')
	#predicting testData & finding max prediction
	predictions = model.predict(testData)
	predictedClassifications = predictions.argmax(axis=-1)
	#checking classifications
	correctClassifications = 0
	for i in range(len(testKey)):
	  if testKey[i] == predictedClassifications[i]:

```
        correctClassifications += 1
accuracy = (correctClassifications/60)
confMatrix = confusion_matrix(testKey, predictedClassifications)
print("Confusion Matrix:")
print(confMatrix)
classTrue = [i for i in range(len(confMatrix)) for _ in
range(len(confMatrix[i]))]
classPred = [j for i in confMatrix for j in i]
precision = precision_score(classTrue, classPred, average='weighted')
f1 = f1_score(classTrue, classPred, average='weighted')
recall = recall_score(classTrue, classPred, average='weighted')
print("Precision:", precision)
print("F1-score:", f1)
print("Recall: ", recall)
print("Test Accuracy: ", accuracy)
```

Appendix B

Raw Data (1 Second Example)

X Acceleration m/s^2	Y Acceleration m/s^2	Z Acceleration m/s^2	X Gyroscope θ/s^2	Y Gyroscope θ/s^2	Z Gyroscope θ/s^2
2.11	3.37	5.06	0.85	0.53	-0.04
-1.74	-1.50	-3.37	0.83	0.50	-0.07
-1.15	0.09	0.51	0.86	0.51	-0.01
-0.38	0.99	0.30	0.84	0.52	-0.06
-1.47	-1.24	0.44	0.85	0.50	-0.04
2.95	1.45	0.92	0.83	0.53	-0.03